

Pandas & scikit-learn Patterns You'll See This Course

Machine Learning in Geoscience — pre-session reference · read before Session 2

This is not a pandas tutorial. It's a recognition guide: a short list of patterns that show up constantly in the course notebooks. You don't need to memorize these — you just need them to look *familiar* when you see them in code, so your attention goes to the concept (data leakage, forecasting logic) instead of decoding syntax. If a pattern below is new to you, try it once in a scratch cell before the session.

1. Shifting values in time

`series.shift(1)`

Moves every value forward by 1 row — row T now holds what was at $T-1$. This is how you turn "the current value" into "yesterday's value" (a lag feature).

```
s = pd.Series([10, 20, 30, 40])
s.shift(1)
# -> [NaN, 10, 20, 30]
```

`series.shift(-1)`

Negative shift pulls a **future** value backward — row T now holds what will happen at $T+1$. This is how a target/label is built. It is also exactly how leakage sneaks into a *feature* by mistake.

```
s.shift(-1)
# -> [20, 30, 40, NaN]
```

Rule of thumb: shift(+) for inputs, shift(−) only for the target.

2. Rolling windows

`series.rolling(24).mean()`

A moving average over the last 24 rows, computed at each point. By default it looks *backward* (includes the current row and the 23 before it).

`series.shift(1).rolling(24).mean()`

Shift first, *then* roll. This excludes the current row from its own average — so a feature at time T only uses information strictly before T . This single pattern is the mechanical core of "leakage-safe" feature engineering.

<code>shift(1)</code>	-> row T now shows value at T-1
<code>.rolling(24)</code>	-> average the 24 rows ending at T-1
<code>.mean()</code>	-> a single number per row

3. Missing values

`.isna()` / `.dropna()`

`isna()` flags missing entries (True/False); `dropna()` removes rows that contain them.

`.ffill()` vs `.bfill()`

`ffill()` (forward-fill) carries the **last known past value** forward — safe. `bfill()` (back-fill) copies a **future** value backward — this is a leakage source if used on data before a forecast origin.

4. Selecting rows by position

`df.iloc[idx]`

Selects rows by integer position. In this course `idx` is usually not a simple slice like `0:10` — it's an array of row numbers computed earlier (e.g. "all rows belonging to the train block"). Read it as "give me exactly these rows," not "give me a range."

```
train_idx = np.arange(0, 700) # computed elsewhere
X.iloc[train_idx]             # exactly those 700 rows
```

5. Splitting data for time series

`train_test_split(X, y, shuffle=True)`

The default in most ML tutorials. It picks rows **randomly** for train/test. Fine for independent data — wrong for time series, because neighboring hours are correlated, so the model can "peek" at nearby answers.

Manual chronological split

Instead, the course computes index arrays for consecutive time blocks (train = earliest 70%, validation = next 15%, test = final 15%) and selects them with `.iloc[]` — no shuffling, no randomness in the split itself.

6. The scikit-learn fit/transform pattern

`estimator.fit(X_train)`

"Learn" something from data — e.g. a scaler learns the mean/std of each column; a model learns its parameters. **Always fit on training data only.**

`estimator.transform(X_other)`

Apply what was learned to new data, without re-learning. Fitting a scaler on the *full* dataset (train+val+test combined) before splitting leaks information about the future into training — one of the four leakage types you'll see demonstrated in Session 2.

```
scaler = StandardScaler().fit(X_train) # learn only from train
X_train_s = scaler.transform(X_train)
X_test_s = scaler.transform(X_test)    # reuse, don't re-fit
```

`Pipeline([...])`

Chains steps (e.g. scaler → model) into one object, so `fit/transform` happen in the right order automatically. You'll mostly just read these, not build them from scratch yet.

7. Quick correlation / diagnostics

`df.corr()`

Pairwise correlation between all numeric columns — used to spot near-duplicate features (e.g. a 24h lag and a 24h rolling mean carrying almost the same information).

Before Session 2 — a 10-minute self-check

- Open a blank notebook and run each example above on a toy Series/DataFrame you make up.
- If `shift(1).rolling(24).mean()` makes sense to you without re-reading it, you're ready.
- If something above is still unclear, that's fine — flag it at the start of the session instead of getting stuck mid-lecture.

Machine Learning in Geoscience — Session 2 pre-read. This sheet covers recognition only; the course notebooks are where these patterns get used for real, with full explanations in context.